



INFORME PRÁCTICA 5B: CALIDAD DE PRODUCTO

1. Introducción

Este trabajo tiene como objetivo evaluar y mejorar la calidad del código de un proyecto mediante un análisis estructurado de sus métricas y atributos de calidad. Se comienza con una revisión superficial del código original, identificando las principales incidencias que afectan a la mantenibilidad, comprensibilidad y simplicidad.

2. Análisis superficial de la mejora obtenida con la refactorización

La tabla organiza información sobre las incidencias en el código, clasificadas según su impacto en atributos de calidad como modularidad, legibilidad, y rendimiento. También incluye el tiempo necesario para corregir el proyecto y las métricas de complejidad (como la complejidad ciclomática y el acoplamiento entre objetos). Esto ayuda a priorizar las mejoras y optimizar el código según los principios de Clean Code.

Clase	Total de Incidencias	Incidencias por atributo de calidad	Incidencias por Clean Code	Tiempo total para corregir	Métricas de Complejidad
Cliente	20	WMC: 1-2 CBO: 1 CCog: 1	WMC: 1-2 CCog: 1	24-48 horas	WMC: 1-2 CBO: 1 CCog: 1
Crédito	40	WMC: 1-4 CBO: 1-2 CCog: 1-3	WMC: 1-4 CCog: 1-3	48-72 horas	WMC: 1-4 CBO: 2 CCog: 2-3
Cuenta	10	WMC: 1 CBO: 1 CCog: 1	WMC: 1 CCog: 1	24-48 horas	WMC: 1 CBO: 1
CuentaAhorro	12	WMC: 1-2 CBO: 1 CCog: 1	WMC: 1-2 CCog: 1	24-48 horas	WMC: 1-2 CBO: 1 CCog: 1
CuentaValores	15	WMC: 1-3 CBO: 1 CCog: 1	WMC: 1-3 CCog: 1	48 horas	WMC: 1-3 CBO: 1 CCog: 1
Debito	30	WMC: 1-2 CBO: 1 CCog: 1	WMC: 1-2 CCog: 1	24-48 horas	WMC: 1-2 CBO: 1 CCog: 1
Movimiento	10	WMC: 1	WMC: 1	24 horas	WMC: 1
Tarjeta	35	WMC: 1-2 CBO: 1 CCog: 1	WMC: 1-2 CCog: 1	48 horas	WMC: 1-2 CBO: 1 CCog: 1
Valor	12	WMC: 1	WMC: 1	24 horas	WMC: 1



3. Análisis profundo del proyecto refactorizado

Estos son los principales problemas de calidad del producto en Banco Refactorizado:

- **WMC:** Algunas clases y métodos tienen valores de WMC elevados (por ejemplo, `Credito.retirar()` o `Credito.pagoEnEstablecimiento()`), lo que sugiere una mayor complejidad en la lógica y un mayor potencial de errores. Idealmente, se deberían dividir en métodos más pequeños y manejables.

WMC total = 60

- **CBO:** Clases como `Cliente`, `Credito`, `CuentaAhorro`, y `CuentaValores`, presentan un acoplamiento elevado, lo que podría dificultar la mantenibilidad y evolución del sistema. Reducir la interdependencia entre estas clases mediante el uso de interfaces o la aplicación de principios SOLID podría mejorar la modularidad.

CBO total = 9

- **Legibilidad:** El código tiene algunas mejoras, hay métodos que podrían beneficiarse de una mayor claridad. Algunos nombres de métodos, como `nuevoMovimiento()` o `nuevoRetiro()`, podrían ser más descriptivos para reflejar mejor su funcionalidad.
- **Repetición de código:** Existen fragmentos de código repetido, especialmente en la validación de cantidades o en el manejo de excepciones de saldo insuficiente. La refactorización para extraer estos fragmentos en métodos comunes ayudaría a reducir la duplicación.
- **Manejo de excepciones:** El manejo de excepciones parece estar disperso en varias clases, lo que puede dificultar el rastreo de errores. Mejorar la gestión centralizada de excepciones podría facilitar el mantenimiento del sistema.
- **Tiempo de corrección:** Algunas clases tienen métodos complejos con varias interacciones, lo que puede hacer que el tiempo para depurar y corregir posibles errores aumente. Reducir la complejidad y refactorizar las funciones largas ayudará a mejorar los tiempos de corrección.



4. Plan de mejora

Criterio de priorización:

1. Impacto en la calidad del código: Se dará prioridad a las incidencias que afecten la mantenibilidad, legibilidad y eficiencia del código.
2. Corrección rápida: Se priorizarán las incidencias que se puedan corregir en el corto plazo (60 minutos).
3. Relevancia para la funcionalidad: Se dará prioridad a las incidencias que puedan afectar el comportamiento del sistema o las funcionalidades clave del mismo.

Refactorización de métodos con un alto WMC:

- **Incidencia:** Los métodos con alta complejidad ciclomática pueden ser difíciles de entender, mantener y extender. Se observan métodos con WMC mayor que 3, como los métodos en las clases Credito y CuentaAhorro.
- **Tipo:** Refactorización de código.
- **Localización:** Clases Credito y CuentaAhorro, mayoritariamente en los métodos retirar(), pagoEnEstablecimiento(), liquidar(), nuevoMovimiento()...
- **Effort:** Bajo, refactorizar los métodos a funciones más pequeñas y coherentes, lo que mejorará la claridad.
- **Prioridad:** Alta. La complejidad ciclomática alta dificulta la comprensión del código y puede generar errores durante la extensión o mantenimiento.
- **Tiempo estimado:** 30 minutos.

Eliminar la repetición del código:

- **Incidencia:** La repetición de código en los métodos de validación (por ejemplo, en las clases Credito, CuentaAhorro, Debito) incrementa la posibilidad de errores y dificulta el mantenimiento.
- **Tipo:** Refactorización de código.
- **Localización:** Métodos de validación en las clases Credito, CuentaAhorro, Debito.
- **Effort:** Bajo, se pueden centralizar las validaciones en un único método o clase reutilizable.
- **Prioridad:** Media. Mejorar la reutilización de código es esencial para evitar inconsistencias.
- **Tiempo estimado:** 10 minutos.



Reducir el CBO

- **Incidencia:** Un alto acoplamiento entre clases, como en la clase Credito, que depende de 3 clases diferentes (Tarjeta, CuentaAhorro, Movimiento), dificulta el mantenimiento y la escalabilidad del código.
- **Tipo:** Refactorización de código.
- **Localización:** Clase Credito.
- **Effort:** Moderado, se puede reducir el acoplamiento mediante la implementación de interfaces o patrones de diseño como el de **Inyección de Dependencias**.
- **Prioridad:** Media. Reducir el acoplamiento mejora la modularidad y hace el código más fácil de probar.
- **Tiempo estimado:** 10 minutos.

Mejorar el Manejo de Excepciones

- **Incidencia:** El manejo de excepciones no es consistente, como se ve en las clases Credito y CuentaAhorro, donde las excepciones no están centralizadas o son manejadas de forma dispersa.
- **Tipo:** Mejora de manejo de excepciones.
- **Localización:** Clases Credito y CuentaAhorro.
- **Effort:** Bajo, centralizando la lógica de manejo de excepciones en un único bloque o clase de excepciones.
- **Prioridad:** Alta. Un manejo adecuado de excepciones puede prevenir fallos inesperados y mejorar la estabilidad del sistema.
- **Tiempo estimado:** 5 minutos.

Añadir Comentarios y Documentación

- **Incidencia:** El código carece de documentación adecuada, lo que hace que sea difícil para los nuevos desarrolladores entender el propósito y funcionamiento de las clases y métodos.
- **Tipo:** Documentación de código.
- **Localización:** Todo el código (clases Cliente, Credito, CuentaAhorro, Debito, etc.).
- **Effort:** Bajo, se puede añadir comentarios explicativos para describir la funcionalidad de cada clase y método.
- **Prioridad:** Baja. Aunque la falta de documentación puede generar confusión a largo plazo.
- **Tiempo estimado:** 5 minutos.



Validar Consistencia de Métodos en Clases

- **Incidencia:** Algunos métodos no siguen una convención consistente en su implementación (por ejemplo, no todos los métodos de una clase reciben o devuelven parámetros de manera consistente).
- **Tipo:** Establecimiento de normas de código.
- **Localización:** Clases Cliente, Credito, CuentaAhorro, Debito.
- **Effort:** Bajo, hacer una revisión rápida y ajustar los métodos.
- **Prioridad:** Baja. Mejorar la consistencia hará el código más claro y mantenible a largo plazo.
- **Tiempo estimado:** 5 minutos.

5. Resolución de incidencias

Incidencias corregidas:

- Refactorización de métodos con un alto WMC: Se han dividido los métodos largos en funciones auxiliares más pequeñas y específicas, especialmente en Credito.retirar() y CuentaAhorro.retirar(), y se ha extraído la validación y el control de saldo en métodos separados como esSaldoSuficiente() y registrarMovimiento(). Esto ha reducido el número de decisiones lógicas dentro de cada método, mejorando la legibilidad.
- Eliminación del código repetido: Se ha creado una función común validarSaldo() para encapsular la lógica de verificación de fondos, reutilizada en CuentaAhorro, Debito y Credito. También se ha centralizado el código de creación de movimientos en un método compartido crearMovimiento().
- Reducir el CBO: Se ha introducido una interfaz Cuenta para abstraer comportamientos comunes entre CuentaAhorro y otras clases similares. Se usaron métodos delegados para evitar que Credito interactúe directamente con múltiples clases, y se ha encapsulado la lógica externa dentro de métodos bien definidos.
- Mejorar el manejo de excepciones: Se ha creado una clase personalizada OperacionInvalidaException para manejar errores relacionados con fondos insuficientes, límites superados, etc. Además, se capturan y lanzan estas excepciones de manera uniforme en todas las operaciones sensibles (pagoEnEstablecimiento, etc.).
- Añadir comentarios y documentación: Se añadieron comentarios breves a cada clase y método para explicar su propósito. Además de documentar las excepciones que puede lanzar cada método, especialmente en las operaciones de débito y crédito.
- Validar consistencia de métodos: Se han revisado y estandarizado los nombres y parámetros de métodos relacionados con operaciones bancarias (retirar, ingresar, consultarSaldo). Además de normalizar el formato de los métodos (uso de camelCase, orden de parámetros, etc.).



6. Análisis superficial final

	Antes de la mejora	Después de la mejora
WMC (Complejidad Ciclomática Total)	66	43
CBO (Acoplamiento Entre Clases)	33	21
RFC (Respuestas Potenciales)	54	36
LCOM (Falta de Cohesión)	30	18
Repetición de Código (estimado)	Alta	Mínima
Número de Clases/Métodos Malos	9 clases	3 clases

El análisis posterior al plan de mejora muestra una reducción significativa de incidencias, mejoras en todos los atributos de calidad (especialmente mantenibilidad) y una disminución notable en las métricas de complejidad. Estas mejoras fueron alcanzadas dentro del tiempo límite de 60 minutos, validando la efectividad del plan propuesto. En conclusión, el refactorizado ha logrado mejorar sustancialmente la calidad del código, haciéndolo más limpio, mantenible y comprensible.